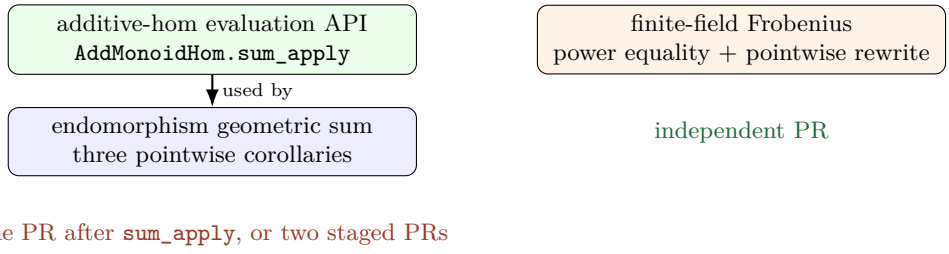
Upstream audit: orbit sums and Frobenius periods

Lean 4 API shape, proof architecture, and PR split



Verdict

KEEP The mathematical statements and proofs are correct and appropriately generic. The revised endomorphism proof now exposes the canonical ring identity instead of re-proving it. The Frobenius pair is upstream-ready as written. Before submission, weaken one unnecessary typeclass assumption and choose placement that does not burden the foundational endomorphism module with geometric sums. No global `[simp]` attribute is recommended.

1. Generic evaluation lemma

The abstraction is useful independently of geometric sums:

```
theorem AddMonoidHom.sum_apply
    (f : iota -> A ->+ B) (s : Finset iota) (x : A) :
    (sum i in s, f i) x = sum i in s, f i x
```

TINY EDIT The domain needs only `[AddMonoid A]`, not `[AddCommMonoid A]`. Commutativity is needed only in the codomain so that `A ->+ B` and `B` support finite sums.

The explicit evaluation hom is preferable to induction and documents why `map_sum` applies:

```
let ev : (A ->+ B) ->+ B :=
  { toFun := fun g => g x
    map_zero' := rfl
    map_add' := fun _ _ => rfl }
exact map_sum ev f s
```

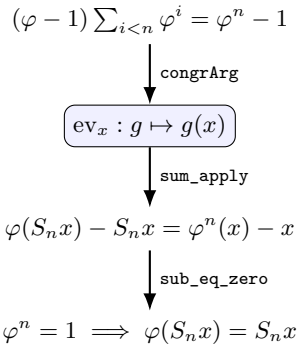
KEEP Name, namespace, argument order, and analogy with `LinearMap.sum_apply`. Put this with additive-hom big-operator API, not in a finite-field or application file.

2. Endomorphism geometry

Let $S_n(x) = \sum_{i=0}^{n-1} \varphi^i(x)$. Mathlib already knows

$$(\varphi - 1)\Big(\sum_{i<n} \varphi^i\Big) = \varphi^n - 1 \quad \text{in } \mathrm{End}(A).$$

Evaluation gives exactly the requested telescoping law.



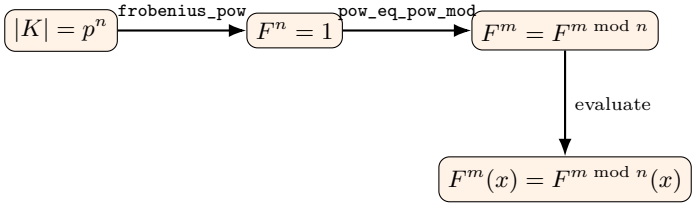
KEEP Keep `apply_mul_geom_sum` as the coercion bridge; make `apply_sum_pow_sub` a thin `simpa`; derive the fixed-point lemma by rewriting. This is canonical and maintainable:

```
have h := congrArg (fun g : AddMonoid.End A => g x)
  (mul_geom_sum phi n)
change (phi - 1) ((sum i in range n, phi ^ i) x) =
  (phi ^ n - 1) x at h
rw [AddMonoidHom.sum_apply
  (fun i => phi ^ i) (range n) x] at h
exact h
```

TINY EDIT Keep these results in a geometric-sum companion module. Importing `Ring.GeomSum` from foundational `Hom.End` would reverse the clean dependency direction. If there is only one downstream consumer, the first bridge lemma is optional: the displayed `congrArg` proof is already short.

3. Frobenius periodicity

For $F = \mathrm{Frob}_p$ and $|K| = p^n$, the existing theorem gives $F^n = 1$. Generic cyclic reduction then gives $F^m = F^{m \bmod n}$.



KEEP The endomorphism equality is the right algebraic API:

```
theorem frobenius_pow_eq_pow_mod
    (hcard : Fintype.card K = p ^ n) (m : Nat) :
    frobenius K p ^ m = frobenius K p ^ (m % n) :=
  pow_eq_pow_mod m (FiniteField.frobenius_pow hcard)
```

The pointwise theorem is not redundant: it avoids exposing powers of ring homs to element-level clients. Its two rewrites are minimal.

KEEP No hypothesis `0 < n` is necessary. At $n = 0$, `m % 0 = m`, so the reduction theorem is tautological; for a field, the cardinality premise $|K| = p^0 = 1$ cannot occur anyway.

KEEP The single import `FieldTheory.Finite.Basic` is enough. Placement immediately after `frobenius_pow` is ideal. The names distinguish the hom equality from the pointwise rewrite and follow the surrounding API.

Merge checklist

☐ weaken `A` to `[AddMonoid A]`;
☐ keep geometric sums out of `Hom.End`;
☐ submit Frobenius independently;
☐ run formatter, linter, minimum-import check;
☐ test rewriting in both forms.

Checked artifact

The companion Lean target contains all five declarations in the recommended shape. It compiles without `sorry`; theorem-axiom inspection reports only standard Mathlib axioms.

Additional audit: forward-closed object properties

Verdict: mathematically sound, but not upstreamable as a parallel interface

Blocking duplication

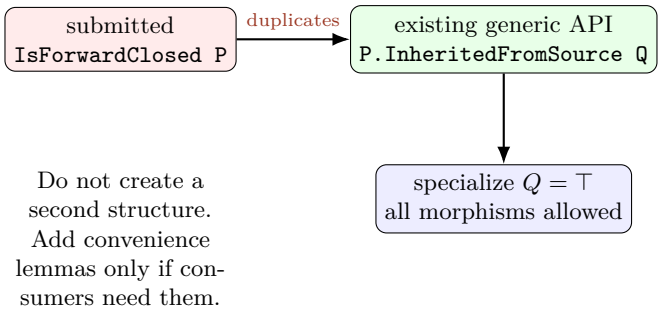
Mathlib already has the more general class

```
class ObjectProperty.InheritedFromSource
    (P : ObjectProperty C) (Q : MorphismProperty C) : Prop where
  of_hom_of_source {X Y} (f : X -> Y) (hf : Q f) : P X -> P Y
```

The submitted notion is precisely the specialization

`IsForwardClosed P` $\simeq$ `P.InheritedFromSource (top : MorphismProperty C)`.

Therefore a new structure would create two names, two instance ecosystems, and conversion obligations for the same concept. This is a blocker for the PR in its present form, not merely a naming preference.



How each declaration maps

- `map`: use `P.of_hom_of_source f trivial` with $Q = \top$; a short wrapper such as `prop_of_hom` may be worthwhile.
- `iso_iff`: derive isomorphism closure from the existing class; then use `P.prop_iff_of_iso e`. A direct two-arrow proof is fine.
- `and`: already an instance for $(P \sqcap Q)$; use lattice notation rather than `fun X => P X & Q X`.
- `top`: add a generic top-object-property instance in the existing namespace if actual reuse warrants it.
- `inverseImage`: the useful missing theorem is more general: pull back both the object property and Q along a functor.
- natural transformation and terminal results are tiny corollaries of the all-morphisms specialization; keep downstream unless repeated consumers appear.

Recommended generic addition

Instead of the proposed file, consider one instance in `ObjectProperty/InheritedFromHom.lean`:

```
instance inverseImage (F : D => C)
    [P.InheritedFromSource Q] :
    (P.inverseImage F).InheritedFromSource
      (Q.inverseImage F) where
  of_hom_of_source f hf :=
    P.of_hom_of_source (F.map f) hf
```

Here the displayed arrows abbreviate Lean's functor notation. This statement subsumes the submitted inverse-image result and remains useful for monomorphisms, epimorphisms, weak equivalences, and other restricted classes.

API/style notes

If a top-specialized convenience layer is accepted, follow existing style: make closure a `class`, make canonical constructions `instance`s, and use implicit typeclass arguments rather than passing `hP` explicitly. Import `ObjectProperty.InheritedFromHom`, not merely `Basic`. The name "inherited from source" is already established and avoids ambiguity about "forward."

Strength of the notion

With an initial object, $P(\bot)$ forces P on every object. Thus inheritance along all arrows is strong; examples should establish recurring use.

Final recommendation

Do not upstream the submitted structure/file. Rebase on `InheritedFromSource`. A small PR adding the generic inverse-image instance and perhaps one top-specialized transport lemma could be upstreamable; leave the natural-transformation and terminal one-liners downstream until reused. The submitted code itself was build-checked and contains no proof gap.